

Pre-Processor Operators

The C preprocessor offers the following operators to help create macros:



Macro Continuation (\)

Continue macros that are too long for a single line



Stringize (#)

Convert macro parameters into string constants



Token Passing (##)

Combine two arguments into a single token



Defined()

Determine if an identifier is defined using `#define`

1/9

The Macro Continuation (\) Operator

A macro is normally confined to a single line. The macro continuation operator (\) is used to continue a macro that is too long for a single line.



Example:

```
#define message_for(a, b) \  
printf("#a " and " #b ": We love you!\n")
```

The backslash (\) at the end of the line tells the preprocessor that the macro definition continues on the next line. This helps improve readability for long macro definitions.

The Stringize (#) Operator

The stringize or number-sign operator (#), when used within a macro definition, converts a macro parameter into a string constant. This operator may be used only in a macro having a specified argument or parameter list.



Example:

```
#include <stdio.h>

#define message_for(a, b) \
printf(#a " and " #b ": We love you!\n")

int main(void)
{
    message_for(Carole, Debra);
    return 0;
}
```

Output:

```
Carole and Debra: We love you!
```

The Token Passing (##) Operator

The token-pasting operator (`##`) within a macro definition combines two arguments. It permits two separate tokens in the macro definition to be joined into a single token.



Example:

```
#include <stdio.h>

#define tokenpaster(n) printf ("token" #n " = %d", token##n)

int main(void)
{
    int token34 = 40;
    tokenpaster(34);
    return 0;
}
```

Output:

```
token34 = 40
```

It happened so because this example results in the following actual output from the preprocessor:

```
printf ("token34 = %d", token34);
```

This example shows the concatenation of token##n into token34 and here we have used both stringize and token-pasting.

✓ The Defined() Operator

The preprocessor defined operator is used in constant expressions to determine if an identifier is defined using `#define`. If the specified identifier is defined, the value is true (non-zero). If the symbol is not defined, the value is false (zero).



Example:

```
#include <stdio.h>

#if !defined (MESSAGE)
#define MESSAGE "Well-done!"
#endif

int main(void)
{
    printf("Here is the message: %s\n", MESSAGE);
    return 0;
}
```

Output:

```
Here is the message: Well-done!
```

Reading from other Files using #include

The preprocessor directive `#include` is an instruction to read in the entire contents of another file at that point. This is generally used to read in header files for library functions. Header files contain details of functions and types used within the library.



Syntax:

```
#include <filename.h>  
or  
#include "filename.h"
```

The contents of the file "filename.h" to be read, parsed, and compiled at that point. The difference between the using of `#` and `"` is that, where the preprocessor searches for the filename.h.

- For the files enclosed in `< >` (less than and greater than symbols) the search will be done in standard directories (include directory) where the libraries are stored.
- For files enclosed in `"` (double quotes) search will be done in "current directory" or the directory containing the source file.

More about #include

Therefore, "" is normally used for header files you've written, and # is normally used for headers which are provided for you (which someone else has written).

Library header file names are enclosed in angle brackets, < >. These tell the preprocessor to look for the header file in the standard location for library definitions. This is /usr/include for most UNIX systems. And c:/tc/include for turbo compilers on DOS / WINDOWS based systems.



Use of #include for the programmer in multi-file programs:

Certain information is required at the beginning of each program file. This can be put into a file by name "globals.h" and included in each program file by the following line:

```
#include "globals.h"
```

If we want to make use of inbuilt functions related to input and output operations, no need to write the prototype and definition of the functions. We can simply include the file by writing:

```
#include <stdio.h>
```

Placing common declarations and definitions into header files means that if they always change, they only have to be changed in one place, which is a much more feasible system.



Conditional Selection of Code using **#ifdef**

The preprocessor has a conditional statement similar to C's if-else. It can be used to selectively include statements in a program. The commands for conditional selection are; **#ifdef**, **#else** and

#endif.



Example:

```
/* Program to find maximum of 2 numbers using #ifdef*/
#include <stdio.h>
#define TWO

main()
{
    int a, b, c;
    clrscr();
#ifdef TWO
{
    printf("\n Enter two numbers: \n");
    scanf("%d %d", &a,&b);
    if(a>b)
        printf("\n Maximum of two numbers is %d", a);
    else
        printf("\n Maximum is of two numbers is %d", b);
}
#endif
}
/* end of main*/
```

Output:

```
Enter two numbers: 33 22
Maximum of two numbers is 33
```

8/9



Using #ifdef for Different Systems

Conditional selection is rarely performed using #define values. This is often used where two different computer types implement a feature in different ways. It allows the programmer to produce a program, which will run on either type.



Example:

```
#include <stdio.h>
main()
{
#ifdef HP
{
```

```

printf("This is a HP system \n");
.....
..... /* code for HP systems*/
}
#endif
#ifdef SUN
{
printf("This is a SUN system \n");
..... /* code for SUN Systems */
}
#endif
}

```

Using #ifdef to temporarily remove program statements:

#ifdef also provides a useful means of temporarily "blanking out" lines of a program. The lines in the program are preceded by #ifdef NEVER and followed by #endif. Of course, you should ensure that the name NEVER isn't defined anywhere.



Example:

```

#include <stdio.h>
main()
{
.....
#ifdef NEVER
{ .....
..... /* code is skipped */

```

```
#endif  
}
```

